

Echo Guard

Architecture Guide

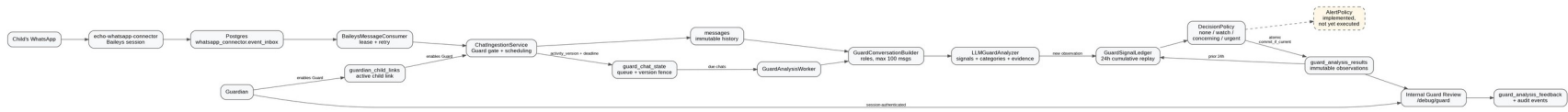
Current implementation map • Guard only

Repository snapshot: gaddyh/echo-v2 • main @ 4020d976 • 5 Oct 2026

Scope This document intentionally excludes Waiting For Me. Shared Echo infrastructure is shown only where Guard depends on it: WhatsApp ingestion, users/connections, persistence, and session authentication.

Guard System Map

Dedicated architecture overview page



End-to-end view: Baileys connector → durable event inbox → ingestion → Guard state/worker → LLM analysis → signal ledger → deterministic decision → review/feedback.

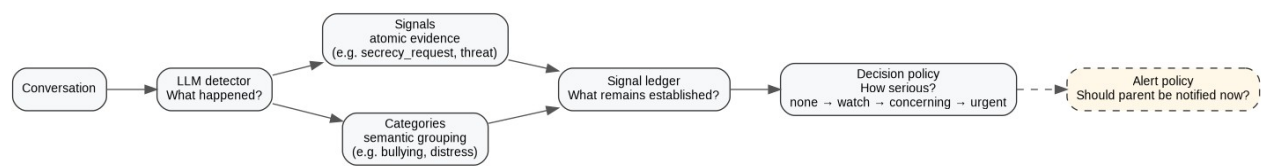
1. The Guard mental model

Echo Guard is a child-safety analysis pipeline built on top of the same durable WhatsApp message spine as Echo. The key design is to separate detection, accumulated evidence, severity, and notification policy instead of asking one LLM to make the entire product decision.

One-sentence architecture WhatsApp events become durable messages → eligible child chats enter an independent Guard queue → an LLM extracts evidence → deterministic code accumulates evidence and decides severity → immutable results are reviewed and labeled by a human.

Four layers of judgment

Layer	Question	Owner	Current role
Detector	What semantic evidence is present?	LLMGuardAnalyzer	Implemented
Memory	What evidence remains established across recent analyses?	GuardSignalLedger	Implemented; 24h replay window
Severity	How serious is the established evidence?	DefaultDecisionPolicy	Implemented, deterministic
Notification	Should a parent be alerted now?	DefaultAlertPolicy	Policy exists; execution not wired



2. Entry into Guard: who is analyzed?

Guard does not analyze every Echo user. At ingestion time, ChatIngestionService checks whether Guard shadow mode is enabled and whether the WhatsApp owner is linked as an active child through guardian_child_links. That gate is what diverts the chat into Guard scheduling.

- A guardian↔child relationship is persisted in guardian_child_links.
- A Guard-enabled child can include group chats even when the ordinary ingestion configuration is private-only.
- The Echo bot's own service chat is explicitly excluded to avoid self-referential analysis.
- The original WhatsApp message is still stored in the shared immutable messages table; Guard owns its own queue state in guard_chat_state.

Baileys path

With the Baileys provider selected, Echo does not receive child WhatsApp events directly from the socket. The separate echo-whatsapp-connector writes normalized events to its durable PostgreSQL event inbox. Echo's BaileysMessageConsumer claims those rows with a lease, resolves the Echo user/connection, and passes a provider-neutral message event into ChatIngestionService.

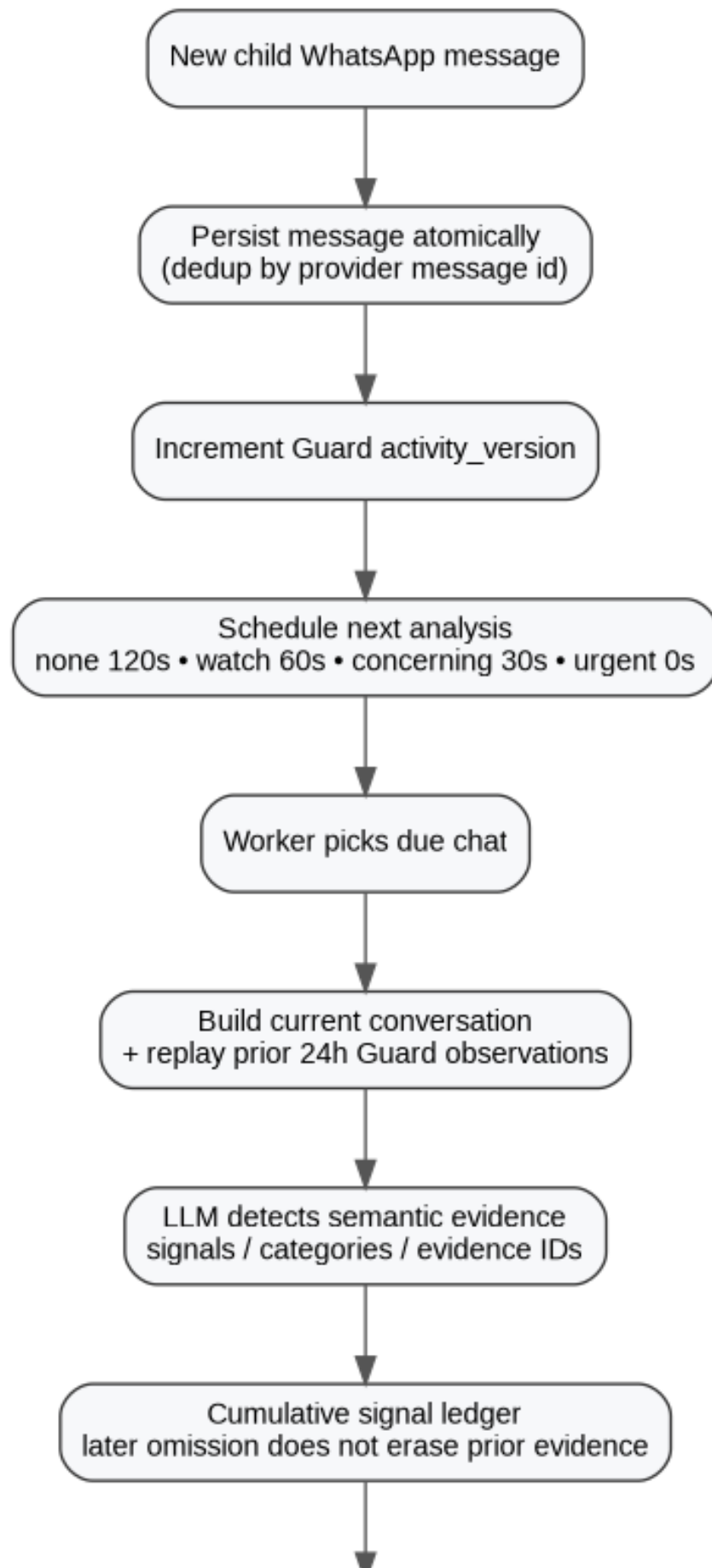
Reliability boundary The connector owns WhatsApp session durability. Echo owns product semantics. The durable event_inbox is the boundary between them.

3. Guard scheduling is state-aware

Every new Guard message increments a per-chat activity_version and computes a new deadline. The scheduling policy intentionally reacts faster when a chat is already risky.

Last decision	Delay after new activity	Meaning
none	120 seconds	Low urgency; batch a little context
watch	60 seconds	Developing pattern; re-check sooner
concerning	30 seconds	Meaningful evidence; shorten latency
urgent	0 seconds	Analyze immediately

If a deadline already exists, the policy keeps the earlier of the existing deadline and the newly computed one. It also persists a human-readable next_analysis_reason so the review UI can explain why the system ran when it did.



Why the version fence matters

LLM work runs outside the database transaction. When the result comes back, `commit_if_current` locks the Guard chat row and checks that `activity_version` still equals the version the worker analyzed. If another message arrived meanwhile, the result is stale and is not allowed to overwrite the newer state.

4. What the Guard analyzer actually sees

`GuardConversationBuilder` loads up to 100 recent persisted messages. It strips real participant identity from the LLM-facing conversation and assigns role labels such as `child`, `other`, `other_2`, and `unknown_participant_1`. Participant identifiers and display names are retained only in diagnostics for internal review.

The LLM output is intentionally detection-only. It must return a strict structured object containing signals, categories, `evidence_message_ids`, `confidence`, and a short reason. The analyzer validates that cited evidence IDs really exist and that every returned category is backed by at least one allowed signal.

Taxonomy v1

Category	Supporting signal family
Suspicious contact	offline knowledge, personal-info/routine/location requests, secrecy, meeting
Child sexual exploitation	age deception, sexual solicitation, intimate-image request, sexual coercion, off-platform migration, secrecy/location/meeting, threat-to-share, blackmail
Harassment / coercion	repeated unwanted contact, boundary violation, threat, coercive demand, blackmail
Distress	help request, fear, hopelessness, self-harm expression
Bullying	repeated targeting, insult/humiliation, group pile-on
Social exclusion	exclusion, coordinated exclusion
Harmful sharing	harmful content sharing, threat to share

5. Guard remembers patterns without asking the LLM to remember

The current analysis is not evaluated in isolation. `GuardAnalysisProcessor` replays prior immutable Guard results from the previous 24 hours into `GuardSignalLedger`, then merges the newest observation. Signals are monotonic inside that replay window: if a later LLM call omits a previously established signal, the omission does not erase it.

Important semantic choice The ledger currently supports accumulation, not explicit retraction. Contradiction/resolution semantics are deliberately deferred until there is a defined product contract.

Deterministic severity policy

Decision	Current deterministic rule
URGENT	Self-harm expression OR blackmail/extortion OR secrecy + meeting request.
CONCERNING	At least two active signals OR at least one active category.
WATCH	At least one active signal, but no concerning/urgent rule.
NONE	No active signals.

This division is a strong architectural property: the LLM detects semantics, while business severity remains inspectable, testable Python.

6. Persistence model



Why results are immutable

Each Guard analysis is stored as an observation keyed by child, chat, target version, and analyzer version. The record keeps not only the decision, signals and evidence IDs, but also model/prompt/analyzer/taxonomy versions and scheduling metadata. That makes later review explainable and allows evaluation by exact algorithm version.

7. Shadow review and the learning loop

The current product loop stops at internal review rather than notifying a parent. /debug/guard is a session-authenticated operator surface. A guardian session can only retrieve Guard results for children linked to that guardian.

- Review cards show current decision, previous decision/version, chat name, categories, signals, confidence, reason, and scheduling timing.
- Evidence is inspectable by message ID with direction, sender, timestamp, and text.
- Filtering is supported by decision, category, chat and date range.
- Feedback has current-state semantics plus an append-only audit history.

Feedback labels

- correct — the Guard result is right.
- false_positive — Guard detected risk that is not really present.
- should_be_stronger — Guard saw something, but under-rated it.
- irrelevant — the result is not useful for the intended product.

8. The alert path: designed, but not yet live

DefaultAlertPolicy already separates severity from notification behavior. It never alerts on none/watch; urgent always alerts; concerning can alert for defined actionable combinations such as suspicious-contact + location/meeting, established bullying, or explicit threat. It also has context hooks for prior alerts, quiet hours, child age and known-contact status.

Current runtime status The policy class exists, but the main application wiring does not yet invoke it to send parent notifications. Guard is therefore presently a shadow-analysis + review system, not a live parent-alert system.

9. Runtime switches that control Guard

Setting	Role	Current implication
WHATSAPP_PROVIDER=baileys	Uses Baileys messaging/provisioning adapters	Needed for the new connector path
BAILEYS_MESSAGE_CONSUMER_ENABLED	Consumes durable Baileys message inbox	Without it, new Baileys messages do not reach Echo ingestion
GUARD_SHADOW_ENABLED	Enables Guard routing for linked children	Without it, Guard state is not scheduled
CHAT_ANALYSIS_ENABLED	Starts analysis worker lifecycle	Guard worker is currently nested under this switch
GUARD_ANALYSIS_POLL_INTERVAL	Guard queue poll cadence	Default in app wiring: 10 seconds
GUARD_SIGNAL_WINDOW_HOURS	Replay window for cumulative signals	Default: 24 hours

10. What exists now vs. what is still missing

Area	Status	Notes
Guardian-child persistence	Implemented	Link model includes consent and safety-enabled timestamps.
Baileys durable ingestion	Implemented	Echo consumers lease event_inbox rows and feed provider-neutral ingestion.
Independent Guard queue	Implemented	guard_chat_state with versioning, scheduling reason and due time.
LLM evidence detection	Implemented	Strict signals/categories/evidence

contract.

Pattern accumulation	Implemented	24h deterministic signal replay.
Severity decision	Implemented	Deterministic policy, not LLM-owned.
Version-fenced commit	Implemented	Stale analyses cannot overwrite newer activity.
Internal review UI	Implemented	Guardian-scoped debug surface with evidence and scheduling diagnostics.
Human feedback + audit trail	Implemented	Current label plus immutable feedback events.
Parent alert execution	Not wired	Alert policy exists; notification orchestration/sending is absent from main runtime.
Signal resolution/retraction	Not defined	Ledger is intentionally monotonic within the replay window.
Production guardian UX	Not yet represented here	Current surface is an internal /debug/guard review workflow.

11. Guard code map

Responsibility	Primary code
Entry/gating	services/chat_ingestion.py
Scheduling	services/guard_schedule_policy.py
Worker	services/guard_analysis_worker.py
Conversation shaping	services/guard_conversation.py
LLM detector	services/guard_analyzer.py
Taxonomy	services/guard_taxonomy.py
Signal memory	services/guard_signal_ledger.py
Severity	services/guard_decision_policy.py
Alert decision	services/guard_alert_policy.py
Processor	services/guard_analysis.py
Domain	domain/guard.py, domain/guard_feedback.py

Repositories	<code>persistence/guard_repositories.py,</code> <code>postgres_guard.py</code>
Schema	<code>migrations 0033–0035</code>
Review	<code>services/guard_review_service.py,</code> <code>app/guard_debug_routes.py</code>

Source basis: current main branch of github.com/gaddyh/echo-v2 at commit `4020d97604b22d88d30a3a0bfbce3b75d8024945`.